

BÀI THỰC HÀNH: PROTOTYPE PATTERN

1. MỤC TIÊU

Sau bài này, sinh viên có thể:

- ✓ Hiểu **Prototype Pattern** là gì
 - ✓ Phân biệt **clone vs new object**
 - ✓ Áp dụng Prototype vào:
 - Clone Order
 - Clone Product
 - Tối ưu performance (tránh khởi tạo lại object nặng)
-

2. MÔ TẢ BÀI TOÁN

Trong hệ thống bán hàng:

👉 Người dùng thường:

- Tạo lại đơn hàng cũ (Re-order)
- Sao chép sản phẩm (Copy product)

👉 Nếu dùng new:

- ✗ Tốn thời gian
- ✗ Code lặp lại
- ✗ Khó maintain

👉 Giải pháp:

- ✓ Dùng **Prototype Pattern**
-

3. Ý TƯỞNG KIẾN TRÚC

```
Frontend (React)
    ↓
POST /api/orders/clone
    ↓
OrdersController
    ↓
Order.Clone()
    ↓
New Order
```

◆ 4. BACKEND – ASP.NET CORE

4.1 Interface Prototype

```
public interface IPrototype<T>
{
    T Clone();
}
```

4.2 MODEL ORDER

```
public class Order : IPrototype<Order>
{
    public int Id { get; set; }
    public List<string> Products { get; set; } = new();
    public decimal Total { get; set; }

    public Order Clone()
    {
        return new Order
        {
            Id = this.Id,
            Products = new List<string>(this.Products),
            Total = this.Total
        };
    }
}
```

4.3 CONTROLLER

```

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private static Order sampleOrder = new Order
    {
        Id = 1,
        Products = new List<string> { "Laptop", "Mouse" },
        Total = 1500
    };

    [HttpGet("original")]
    public IActionResult GetOriginal()
    {
        return Ok(sampleOrder);
    }

    [HttpPost("clone")]
    public IActionResult CloneOrder()
    {
        var cloned = sampleOrder.Clone();

        cloned.Id = new Random().Next(100, 999);

        return Ok(cloned);
    }
}

```

5. FRONTEND – REACT

src/App.js

```

import React, { useState } from "react";
import axios from "axios";

function App() {

    const [original, setOriginal] = useState(null);
    const [clone, setClone] = useState(null);

```

```

    const getOriginal = async () => {
      const res = await
    axios.get("https://localhost:xxxx/api/orders/original");
      setOriginal(res.data);
    };

    const cloneOrder = async () => {
      const res = await
    axios.post("https://localhost:xxxx/api/orders/clone");
      setClone(res.data);
    };

    return (
      <div style={{ padding: 40 }}>
        <h2>Prototype Pattern Demo</h2>

        <button onClick={getOriginal}>Get Original
Order</button>
        <button onClick={cloneOrder}>Clone Order</button>

        <br /><br />

        {original && (
          <div>
            <h3>Original:</h3>
            <pre>{JSON.stringify(original, null, 2)}</pre>
          </div>
        )}

        {clone && (
          <div>
            <h3>Clone:</h3>
            <pre>{JSON.stringify(clone, null, 2)}</pre>
          </div>
        )}
      </div>
    );
  }
}

export default App;

```

6. LUỒNG THỰC THI

1. User click "Get Original"
 2. Server trả Order gốc
 3. User click "Clone"
 4. Server:
 - gọi Clone()
 - tạo object mới
 - trả về
 5. UI hiển thị 2 object khác nhau
-

◆ 7. KẾT QUẢ MONG ĐỢI

Object Id	Products	Total
Original 1	Laptop, Mouse	1500
Clone 234	Laptop, Mouse	1500

👉 Khác Id nhưng giống dữ liệu

◆ 8. SO SÁNH QUAN TRỌNG

Tiêu chí	new object	Prototype
Hiệu năng	✗ chậm	✓ nhanh
Code	✗ lộn	✓ tái sử dụng
Maintain	✗ khó	✓ dễ

◆ 9. BÀI TẬP THỰC HÀNH

🔧 Bài 1

👉 Clone Product

```
public class Product : IPrototype<Product>
{
    public string Name { get; set; }
```

```
    public decimal Price { get; set; }

    public Product Clone()
    {
        return (Product)this.MemberwiseClone();
    }
}
```

Bài 2

Deep Clone OrderItem

- Order có List<OrderItem>
 - Clone toàn bộ object con
-

Bài 3 (Nâng cao)

Clone + chỉnh sửa

- Clone order
- Thêm product mới
- Tính lại total